

University of Applied Sciences Hamburg	Bus Systems	 Hochschule für Angewandte Wissenschaften Hamburg Hamburg University of Applied Sciences
Group No: 1	Lab 3 Room 8.65	Efe Kardes Matriculation No: 2757072
Date: 09/05/2026		Emir Avni Sahin Matriculation No: 2756541
Professor: Ratmus Rettig		Stanislaw Chilecki Matriculation No: 2761057
Function and Characteristics of an Inertial Measurement Cluster STM LSM6DS3		

Report History

Report 1		Date		Remarks
	received			
	checked			
	result	o.k.		
		n.g.		

Prof.

Contents

1	Part 1: Setup	2
1.1	Physical Implementation and Setup Photo	2
1.2	System Architecture Diagrams	2
1.3	Serial Interface Validation	3
2	Part 2: Sensor Configuration	4
2.1	Calibration Data and Physical Orientation	4
2.2	Resolution Characteristics	4
2.3	Sensor Offset and Unexpected Characteristics	6
3	Part 3: Oscilloscope Measurements on the I²C Bus	7
3.1	Experimental Setup	7
3.2	Telegram Analysis and Annotation	8
3.3	Addressing and Bit-Shift Protocol	9
3.4	Physical Layer and Protocol Analysis	10
3.4.1	Data Rate Calculation	10
3.4.2	Physical Layer Comparison	10
4	Part 4: Noise on an Acceleration Sensor	10
4.1	Measurement Method	11
4.2	Raw Measurement Results	11
4.3	Time Series and Histograms	11
4.4	Software Filter	17
4.5	Results After Median Filtering	17
4.6	Comparison Before and After Filtering	22
4.7	Standard Deviation as a Function of ODR	23
4.8	Sensor Offset Calculation	23
4.9	Discussion	24
5	Part 5 – Noise on a yaw rate sensor	24
5.1	Aim of the measurement	24
5.2	Time series and histograms	25
5.2.1	Measurement range: 245 dps	25
5.2.2	Measurement range: 500 dps	26
5.3	Statistical results	28
5.4	Standard deviation as a function of ODR	28
5.5	Sensor offset calculation	29
5.6	Filter documentation	29
5.7	Conclusion	30
6	Filter Implementation Source Code	30

1 Part 1: Setup

This section details the physical hardware connections and validates the data transmission pipeline.

1.1 Physical Implementation and Setup Photo

The sensor is mounted securely within a custom housing to eliminate external vibrations during static characterization. The annotated image below identifies the power, ground, and data line connections.

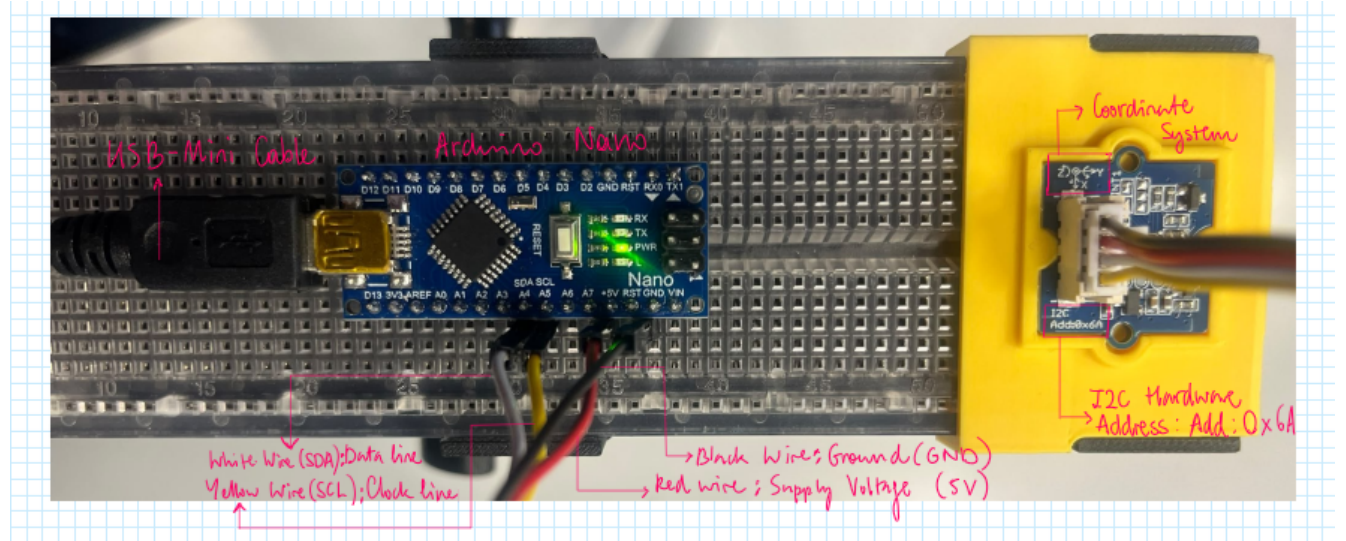


Figure 1: Annotated hardware setup showing the Arduino Nano and LSM6DS3 IMU connections.

1.2 System Architecture Diagrams

The following diagrams illustrate the electrical connections and the logical flow of data from the MEMS sensor to the host computer.

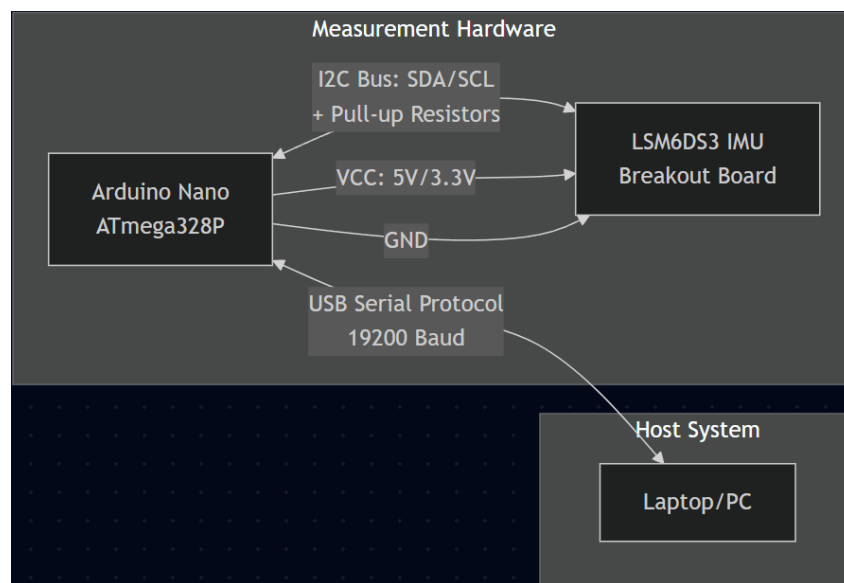


Figure 2: Hardware block diagram.

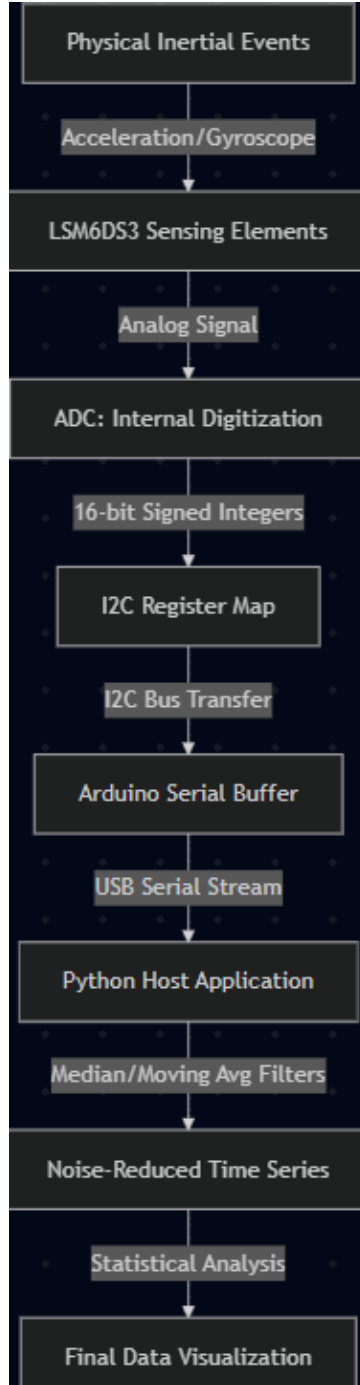


Figure 3: Data-flow diagram detailing the measurement chain.

1.3 Serial Interface Validation

To verify the communication parameters, the serial output was monitored under matched and mismatched baud rate conditions. The Arduino was programmed to transmit data at 19200 bits per second (bps).

When the receiving serial terminal or Python script is synchronized to the correct 19200 baud rate, the data pipeline accurately decodes the transmitted ASCII characters. This results in stable, readable integer outputs representing the raw IMU measurements.

1 16871
 2 16871
 3 16870

```

4 16865
5 16865
6 16865
7 16868
8 16872

```

Listing 1: Correct Data Transmission at Matched Baud Rate (19200 bps)

Because asynchronous serial communication relies on predefined timing intervals rather than a shared clock signal, parameter alignment is critical. When the receiver is misconfigured to 9600 baud, it samples the incoming data stream at half the expected frequency. This misalignment causes framing errors, misinterpreting the start/stop bits and the data payload, which manifests as corrupted, unreadable characters.

```

1 J
2
3 J
4
5 J J J
6
7 J J J J J J
8 J J J J J

```

Listing 2: Corrupted Data Transmission at Mismatched Baud Rate (9600 bps)

2 Part 2: Sensor Configuration

This section evaluates the accelerometer response and resolution across four distinct measurement ranges: $\pm 2g$, $\pm 4g$, $\pm 8g$, and $\pm 16g$.

2.1 Calibration Data and Physical Orientation

To calibrate the z-axis, we mounted the sensor in a custom 3D-printed housing and secured it to a tripod. We established the $+1g$ reference by aligning the z-axis perfectly vertical, counteracting Earth’s gravity. Rotating the sensor 90 degrees placed the z-axis horizontally to record the $0g$ state. Finally, we inverted the sensor 180 degrees to point the z-axis toward the floor, capturing the $-1g$ state.

We recorded 1000 samples per orientation across the four full-scale measurement ranges. The table below presents the mean digital output values (rounded to the nearest integer) for each configuration.

Measurement Range	Output at $+1g$ (LSB)	Output at $0g$ (LSB)	Output at $-1g$ (LSB)
$\pm 2g$	16903	14	-16143
$\pm 4g$	8436	21	-8052
$\pm 8g$	4227	9	-4025
$\pm 16g$	2117	6	-2013

Table 1: Mean digital output values across different measurement ranges and physical orientations.

2.2 Resolution Characteristics

The LSM6DS3 features a 16-bit analog-to-digital converter, providing 65,536 discrete quantization levels. The resolution (the physical value represented by a single Least Significant Bit, or LSB) scales inversely with the chosen measurement range. As the full-scale range expands to capture heavier shocks, each digital step must represent a larger slice of physical acceleration.

We calculate the theoretical resolution by dividing the total physical span by the available 16-bit levels. For example, the $\pm 2g$ range covers a total span of $4g$. Dividing $4g$ by 65,536 yields a sensitivity of exactly 16,384 LSB/g. Taking the inverse provides the resolution per bit. To convert this resolution into m/s^2 , we multiply the g value by standard gravity ($9.81 m/s^2$).

The theoretical resolution characteristics for each tested range are as follows:

- **$\pm 2g$ Range:** 16,384 LSB/g. Resolution = 0.061 mg/LSB or 0.00060 m/s^2 /LSB.
- **$\pm 4g$ Range:** 8,192 LSB/g. Resolution = 0.122 mg/LSB or 0.00120 m/s^2 /LSB.
- **$\pm 8g$ Range:** 4,096 LSB/g. Resolution = 0.244 mg/LSB or 0.00239 m/s^2 /LSB.
- **$\pm 16g$ Range:** 2,048 LSB/g. Resolution = 0.488 mg/LSB or 0.00479 m/s^2 /LSB.

Our measured data strongly validates these theoretical values. We can calculate the actual measured sensitivity by taking half the difference between the $+1g$ and $-1g$ readings (which cancels out the zero-g offset bias). For the $\pm 2g$ range, $\frac{16903 - (-16143)}{2}$ equals a measured sensitivity of 16,523 LSB/g. This is within 0.85% of the theoretical 16,384 LSB/g specification, proving the hardware is functioning correctly and accurately scaling the output data based on the CTRL1_XL register configuration.

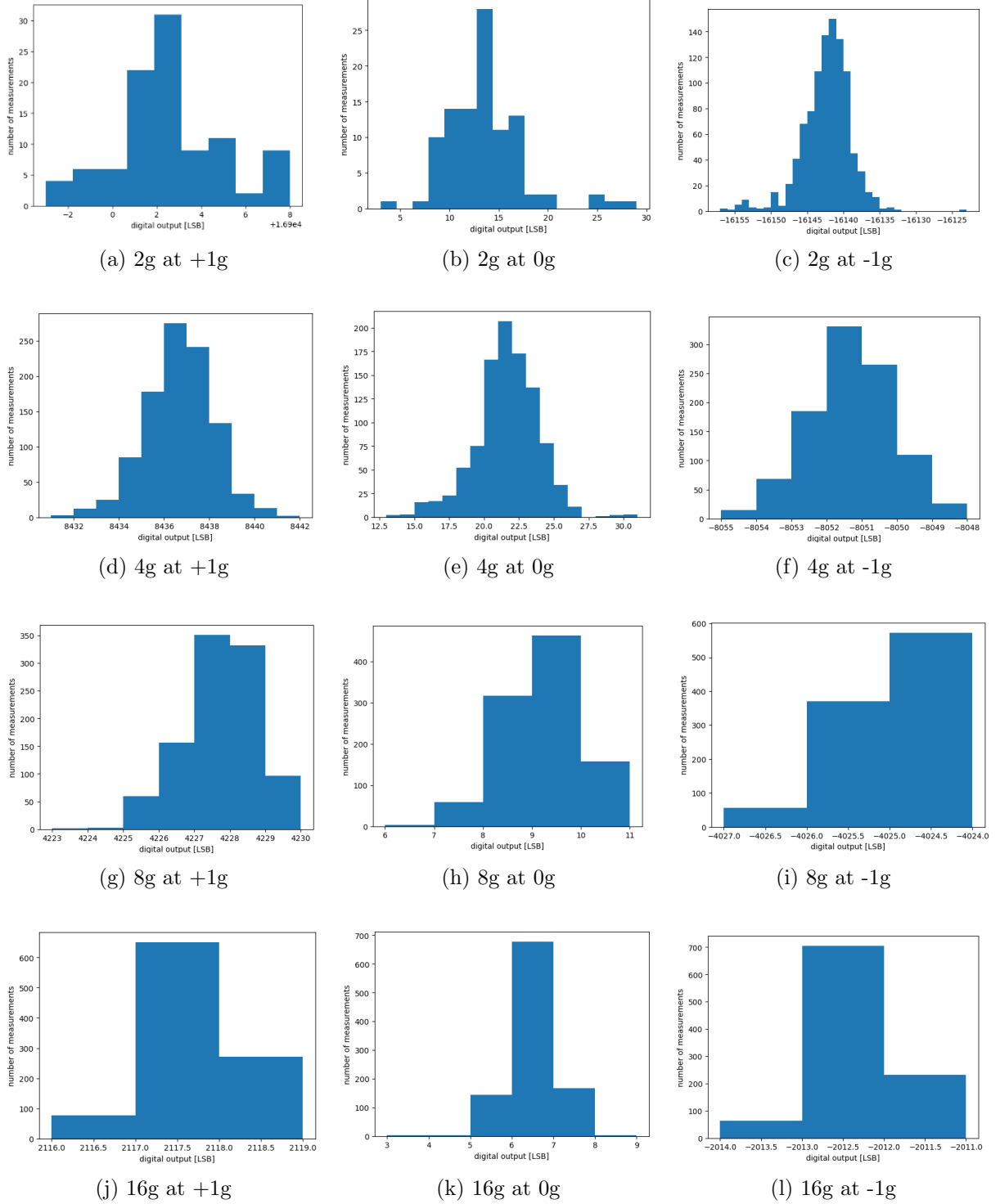


Figure 4: Histograms representing the digital output variance across four measurement ranges (rows) and three gravitational orientations (columns). The horizontal spread visually represents the static noise floor at each configuration.

2.3 Sensor Offset and Unexpected Characteristics

This unexpected characteristic strongly indicates that the tripod was not perfectly level during the +1g and -1g measurements. A small tilt in the setup would introduce a slight projection of the gravity vector, artificially inflating the positive reading and dampening the negative reading, resulting in the mathematically skewed 380 LSB bias. In a real-world software implementation,

the true zero-g bias (e.g., the correctly measured ~ 14 LSB horizontal offset) must be subtracted from all incoming digital readings to properly re-center the accelerometer data around absolute zero.

Method 1: Direct 0g Measurement When the z-axis was placed exactly horizontal (perpendicular to gravity), the sensor output a mean of 13.56 LSB. Using our calculated sensitivity factor (16,384 LSB/g), this translates to an offset of roughly +0.8 mg.

Method 2: Midpoint Calculation Alternatively, we calculate the offset mathematically by finding the midpoint between the inverted physical extremes. The factory zero-g bias should shift both the positive and negative readings equally in one direction.

$$\text{Offset} = \frac{\text{Mean}_{+1g} + \text{Mean}_{-1g}}{2}$$

$$\text{Offset} = \frac{16902.59 + (-16142.82)}{2} = \mathbf{379.88 \text{ LSB}}$$

Unexpected Characteristics: A significant difference exists between these two calculation methods. The direct horizontal measurement implies an offset of ~ 14 LSB, while the mathematical midpoint of the vertical measurements reveals an offset of ~ 380 LSB (≈ 23 mg).

This unexpected characteristic strongly indicates that the tripod was not perfectly level during the $+1g$ and $-1g$ measurements. A small tilt in the setup would introduce a slight projection of the gravity vector, artificially inflating the positive reading and dampening the negative reading, resulting in the mathematically skewed 380 LSB bias.

3 Part 3: Oscilloscope Measurements on the I²C Bus

3.1 Experimental Setup

The I²C physical layer was analyzed using an oscilloscope connected to the SDA and SCL lines of the breakout board, referenced to system ground. Probes were calibrated prior to acquisition. The signals were captured at a time base of 20.0 $\mu\text{s}/\text{div}$.

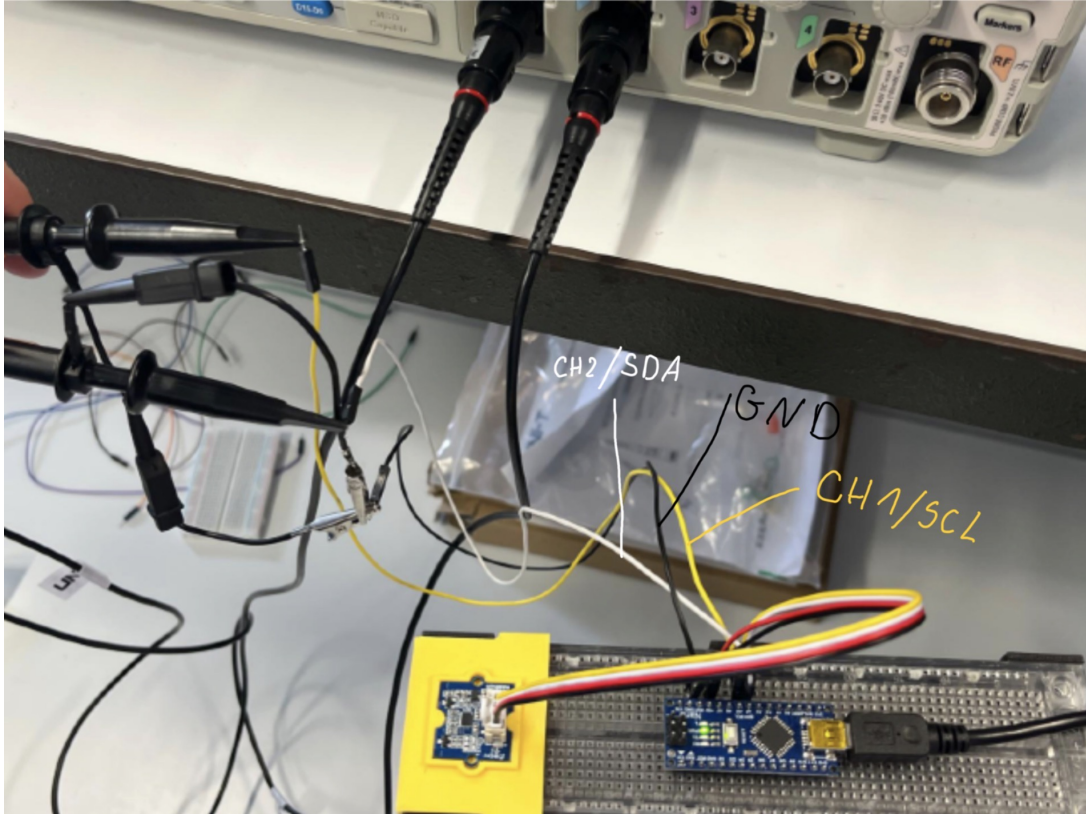


Figure 5: Experimental setup showing Arduino Nano, LSM6DS3 and Oscilloscope connections.

3.2 Telegram Analysis and Annotation

The I²C telegram was decoded by mapping SDA logic levels against SCL rising edges.

- **Start Condition:** SDA transition from High to Low while SCL is High.
- **Binary Stream (MSB First):** 11010100.
- **Hexadecimal Conversion:** $11010100_2 = 0xD4_{16}$.
- **Addressing Logic:** The base address $0x6A$ is shifted left by 1 bit, resulting in $0xD4$. The LSB '0' denotes a Write operation.
- **Acknowledge (ACK):** The 9th bit, where the sensor pulls SDA low.

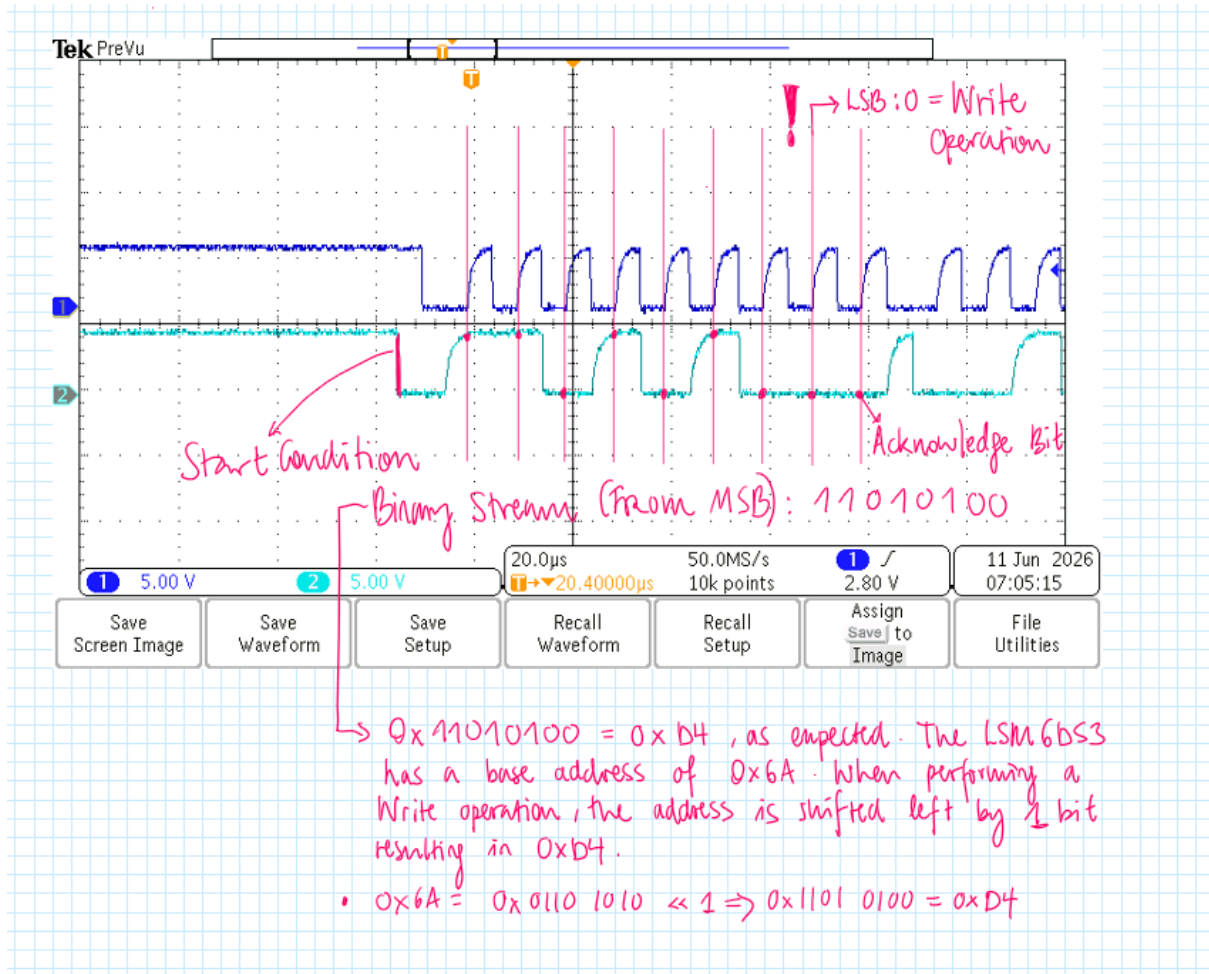


Figure 6: Annotated I²C telegram identifying Start condition, bits 1–8 (Address/RW), and the ACK bit.

3.3 Addressing and Bit-Shift Protocol

- The 7-bit Address Structure:** The LSM6DS3 sensor utilizes a 7-bit device address, which is $0x6A$ (01101010_2). In the I²C protocol, however, a byte (8 bits) is required for the initial transmission.
- The R/W Bit:** To accommodate the 7-bit address within an 8-bit byte, the protocol shifts the address to the left by one position, creating an empty Least Significant Bit (LSB). This LSB serves as the Read/Write indicator bit.
- Bit-Shift Logic:**
 - When the master (Arduino) intends to write data to the sensor, it sets the LSB to '0'.
 - Performing this shift on the base address $0x6A$ ($01101010_2 \ll 1$) results in 11010100_2 , or $0xD4_{16}$.
 - This explains why the first byte observed on the bus is $0xD4$ rather than the base address $0x6A$.
- Communication Significance:** The first byte functions as the "Device Select" phase. It ensures that only the intended sensor on the bus responds to the subsequent clock pulses.

By shifting the address, the master simultaneously transmits the destination identity and the intended operation direction (Write=0, Read=1) in a single, efficient 8-bit transaction.

3.4 Physical Layer and Protocol Analysis

3.4.1 Data Rate Calculation

Based on the oscilloscope time base of $20.0\ \mu\text{s}/\text{div}$, one full clock cycle period (T) was measured to be approximately $10\ \mu\text{s}$ (covering half a division). The data rate (f) is calculated as $f = 1/T$, yielding exactly $100\ \text{kHz}$. This confirms the bus is operating perfectly at the I²C Standard Mode speed. Total raw bits per second are derived by multiplying this frequency by the number of bits successfully clocked during the transmission window.

3.4.2 Physical Layer Comparison

Comparing the measured signals to an ideal I²C physical layer reveals specific deviations:

- **Signal Rise Time:** Unlike the instantaneous transitions of an ideal square wave, the captured signals exhibit rounded, exponential "rise" profiles.
- **Cause of Deviation:** This is due to the bus capacitance interacting with the hardware pull-up resistors (typically $4.7\ \text{k}\Omega$ on standard breakout boards, or the Arduino's internal $\sim 20 - 50\ \text{k}\Omega$ pull-ups). This interaction creates an RC time constant ($\tau = R \times C$) that limits how quickly the signal can reach the high logic threshold. The specific value of these resistors strictly dictates the maximum reliable speed of the bus.
- **Noise and Overshoot:** Minor voltage ringing may be observed during transitions, which is typical for real-world breadboard setups compared to idealized theoretical models.

4 Part 4: Noise on an Acceleration Sensor

This section analyses the noise behaviour of one accelerometer axis of the LSM6DS3 sensor. The accelerometer measurement range was configured to $\pm 2g$. The sensor was kept still during all measurements, so changes in the output signal are interpreted as sensor noise, quantisation effects, small external vibrations, or short measurement disturbances.

The tested Output Data Rate (ODR) settings were:

- $13\ \text{Hz}$
- $104\ \text{Hz}$
- $833\ \text{Hz}$
- $3330\ \text{Hz}$
- $6660\ \text{Hz}$

For every ODR setting, 1000 samples were recorded. The analysed accelerometer axis was the same axis used in Part 2 for the $+1g$ stationary orientation.

4.1 Measurement Method

During the measurements, the sensor was placed in a stable position and was not moved. The Arduino read the accelerometer values from the LSM6DS3 sensor and transmitted the raw digital output values to the computer through the serial interface.

The following statistical values were calculated for each ODR setting:

- mean value,
- median value,
- minimum and maximum value,
- standard deviation,
- sensor offset.

The standard deviation was used as the main indicator of noise. A higher standard deviation means that the measured values are more spread around the mean value, which indicates stronger noise or stronger disturbances.

4.2 Raw Measurement Results

Table 2 shows the statistical results of the raw accelerometer measurements.

Table 2: Raw accelerometer measurement results for different ODR settings.

ODR [Hz]	Samples	Mean [LSB]	Median [LSB]	Min/Max [LSB]	Std. dev. [LSB]
13	1000	16883.60	16884.00	16871 / 16894	3.86
104	1000	16887.37	16887.00	16859 / 16908	7.25
833	1000	16885.58	16882.00	15860 / 21317	161.88
3330	1000	16881.09	16881.00	16640 / 17150	70.85
6660	1000	16890.37	16886.00	16640 / 17151	89.27

At low ODR settings, especially at 13 Hz and 104 Hz, the signal is very stable. The standard deviation is low and the minimum and maximum values are close to the mean value. At higher ODR settings, the standard deviation increases. The 833 Hz measurement shows the highest raw standard deviation because the measurement contains strong outliers. This can be seen from the large difference between the minimum value of 15860 LSB and the maximum value of 21317 LSB.

4.3 Time Series and Histograms

The time series plots were used to check whether the measured signal was stable and whether short spikes or outliers occurred. The histograms were used to compare the distribution of measured values for each ODR setting.

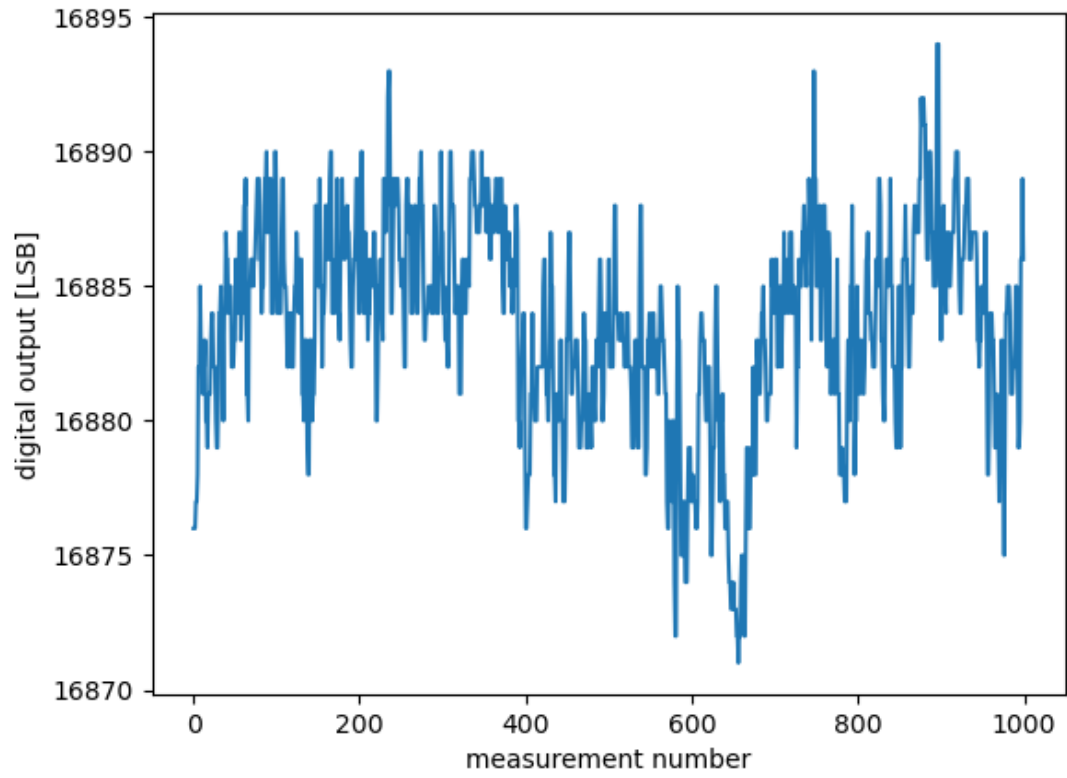


Figure 7: Raw time series for 13 Hz

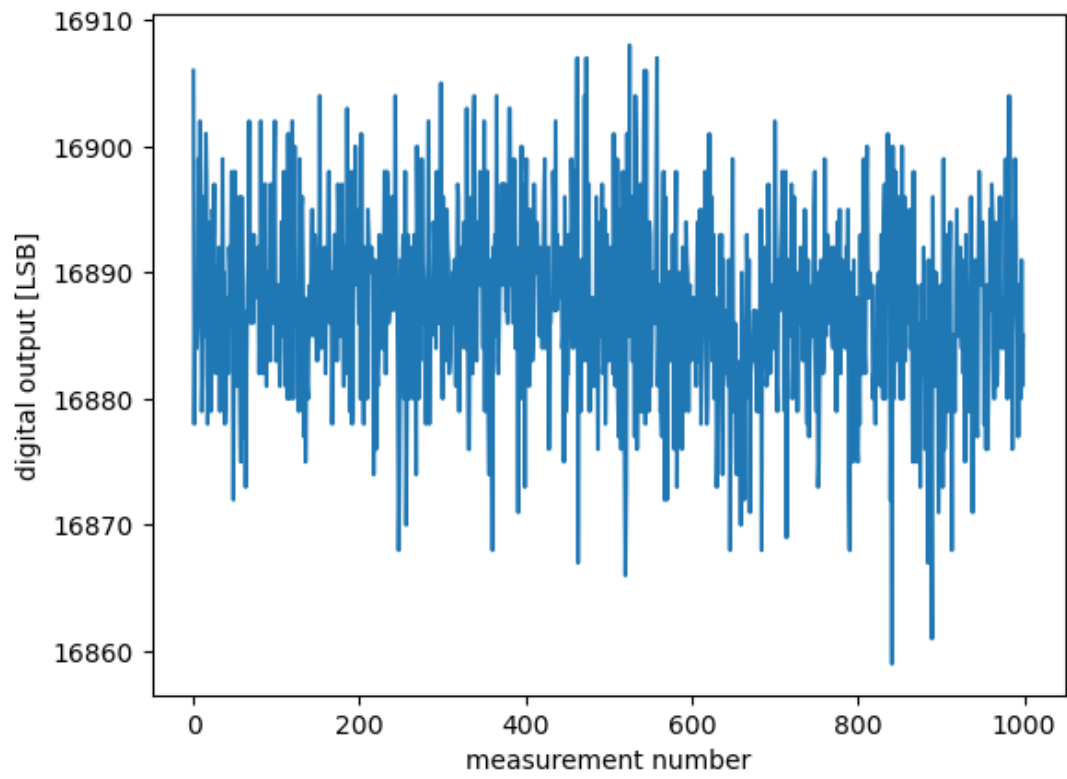


Figure 8: Raw time series for 104 Hz

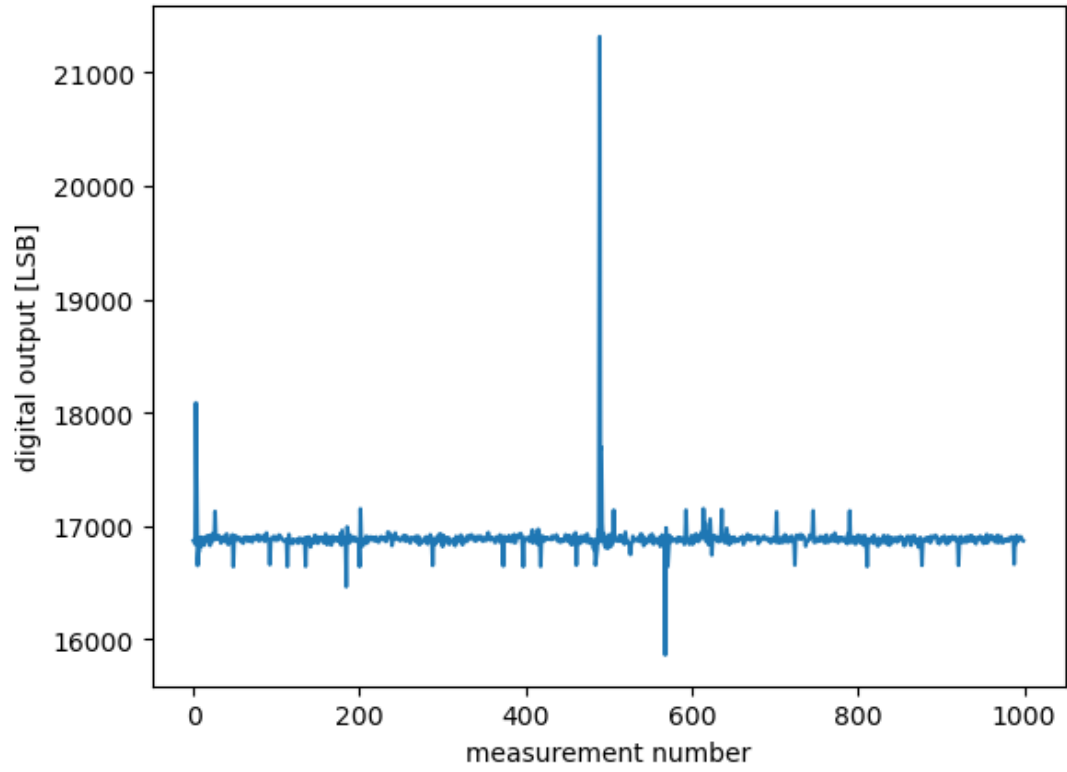


Figure 9: Raw time series for 833 Hz

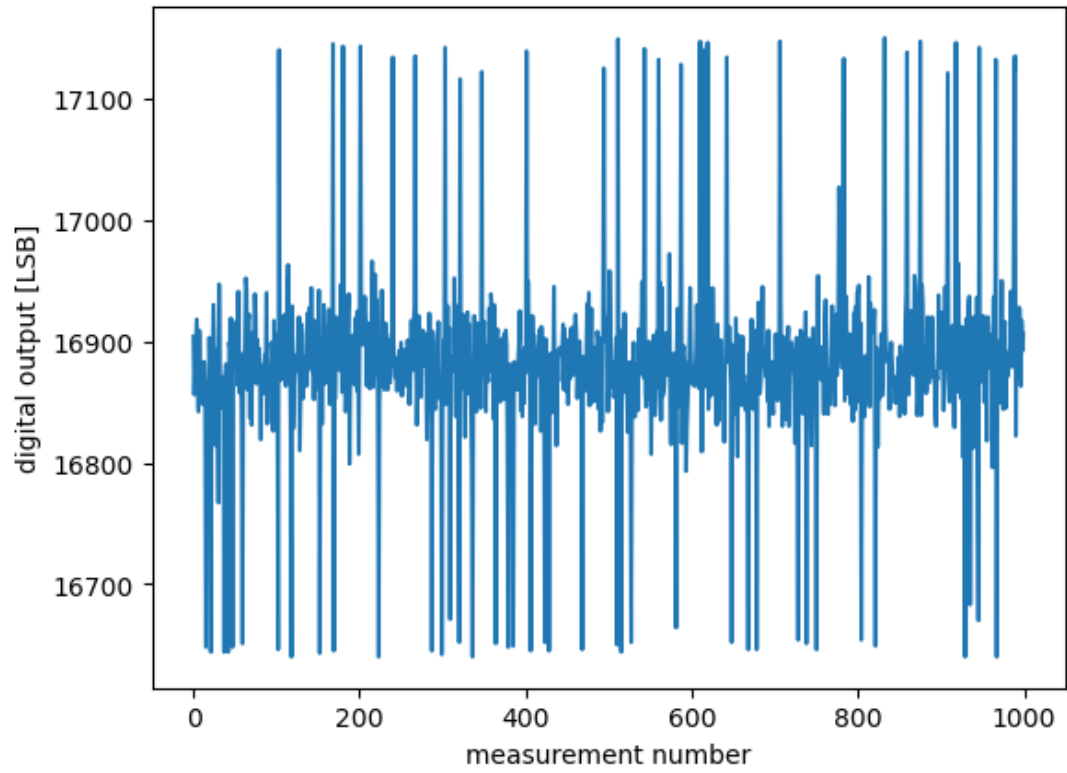


Figure 10: Raw time series for 3330 Hz

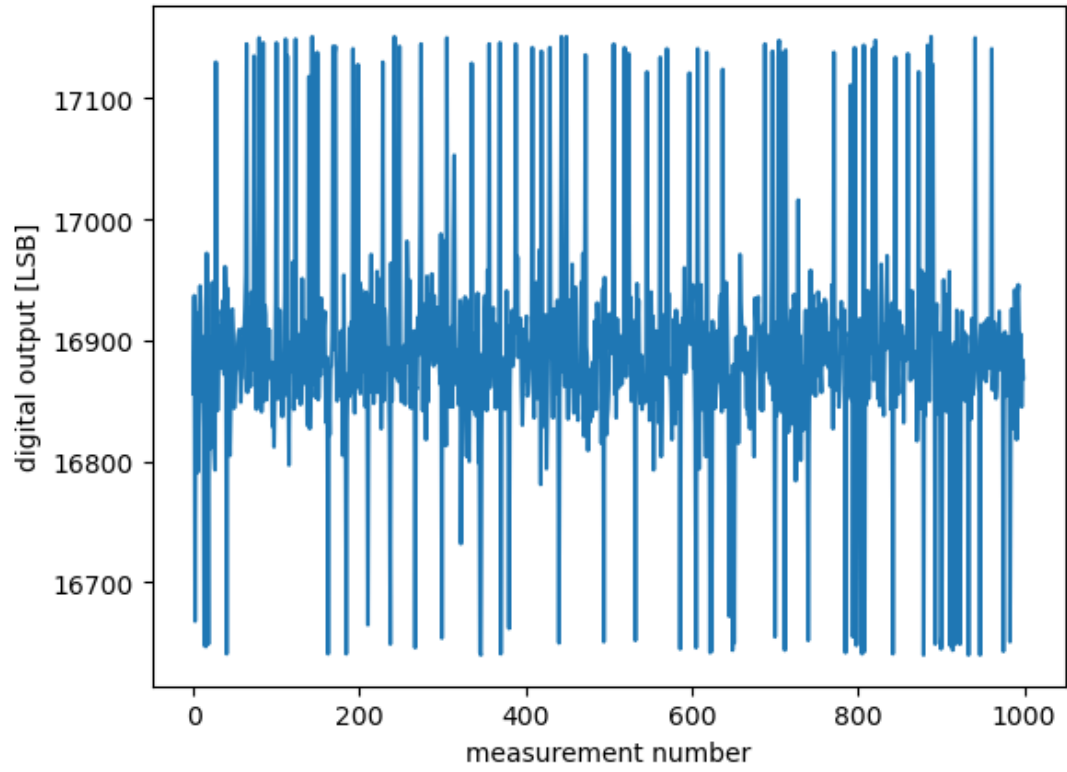


Figure 11: Raw time series for 6660 Hz

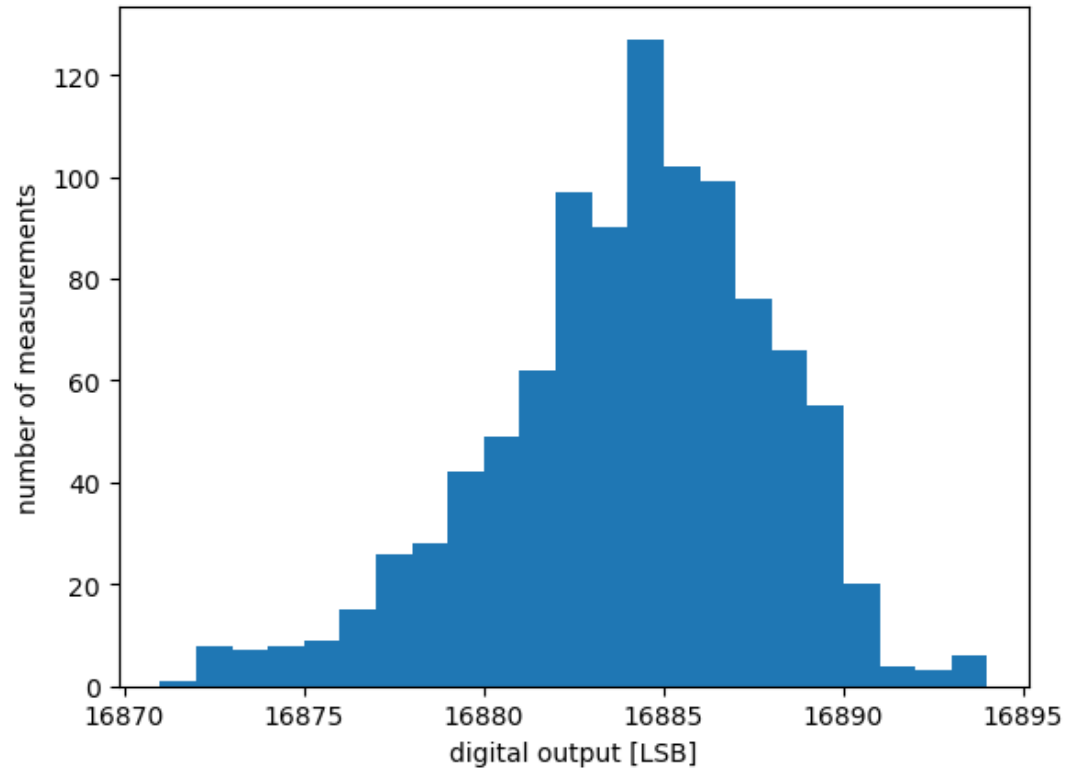


Figure 12: Raw histogram for 13 Hz

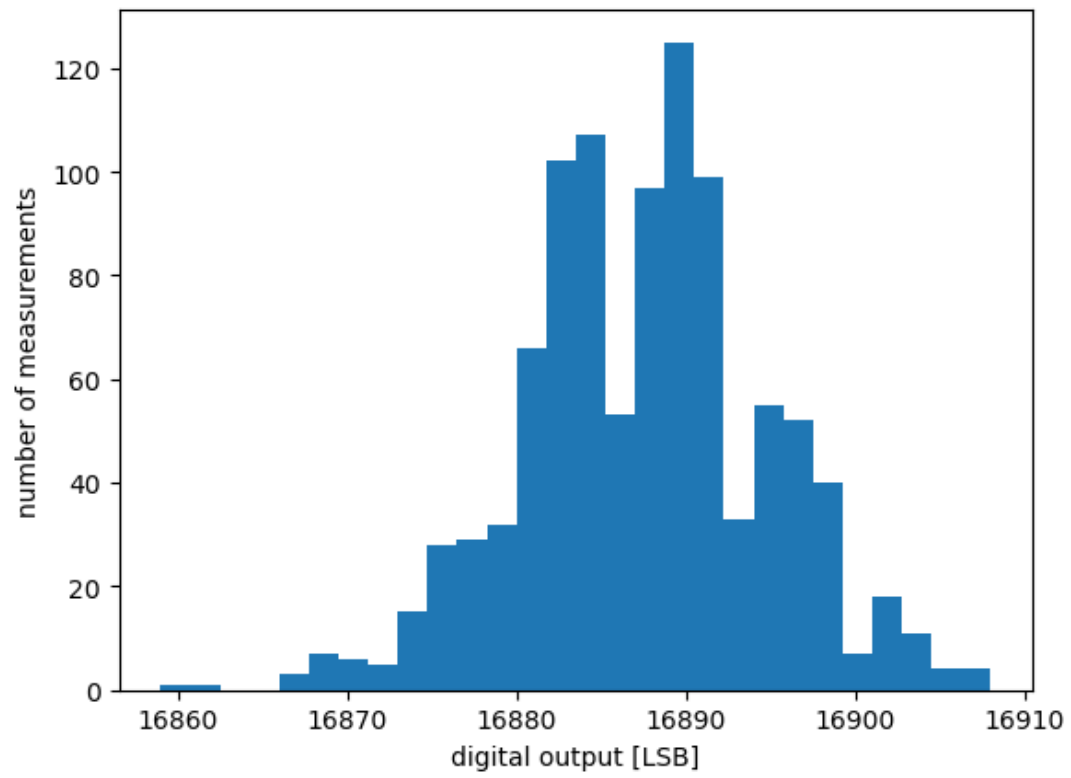
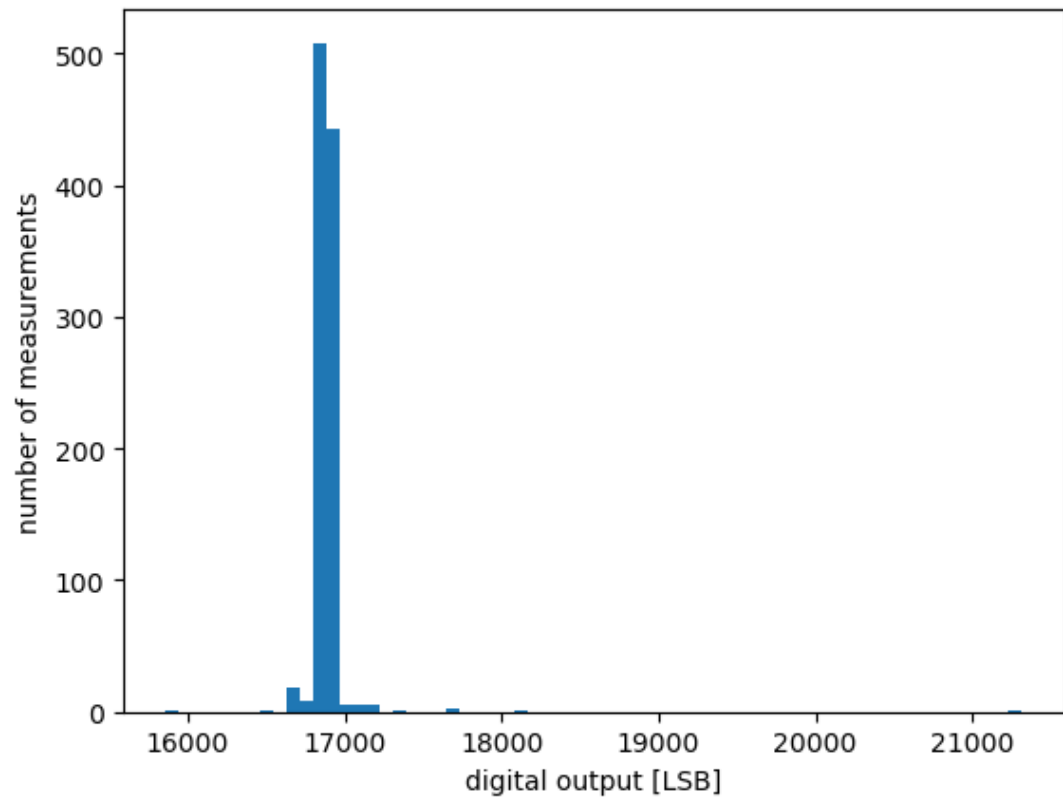


Figure 13: Raw histogram for 104 Hz



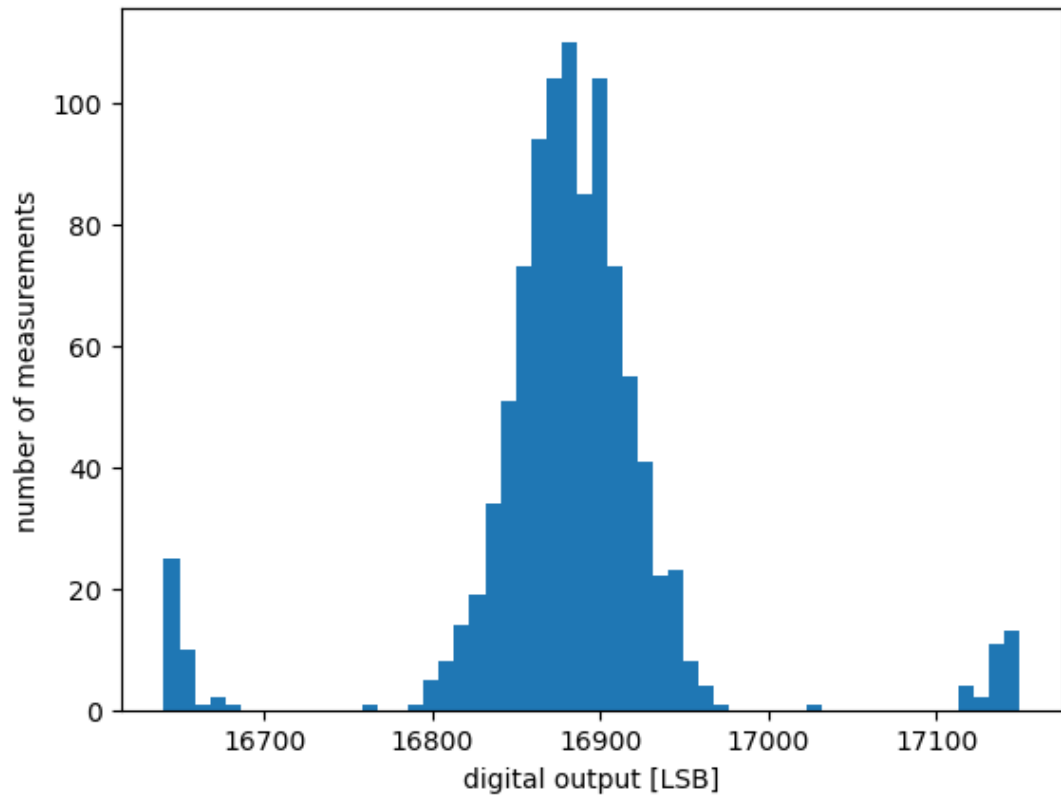


Figure 15: Raw histogram for 3330 Hz

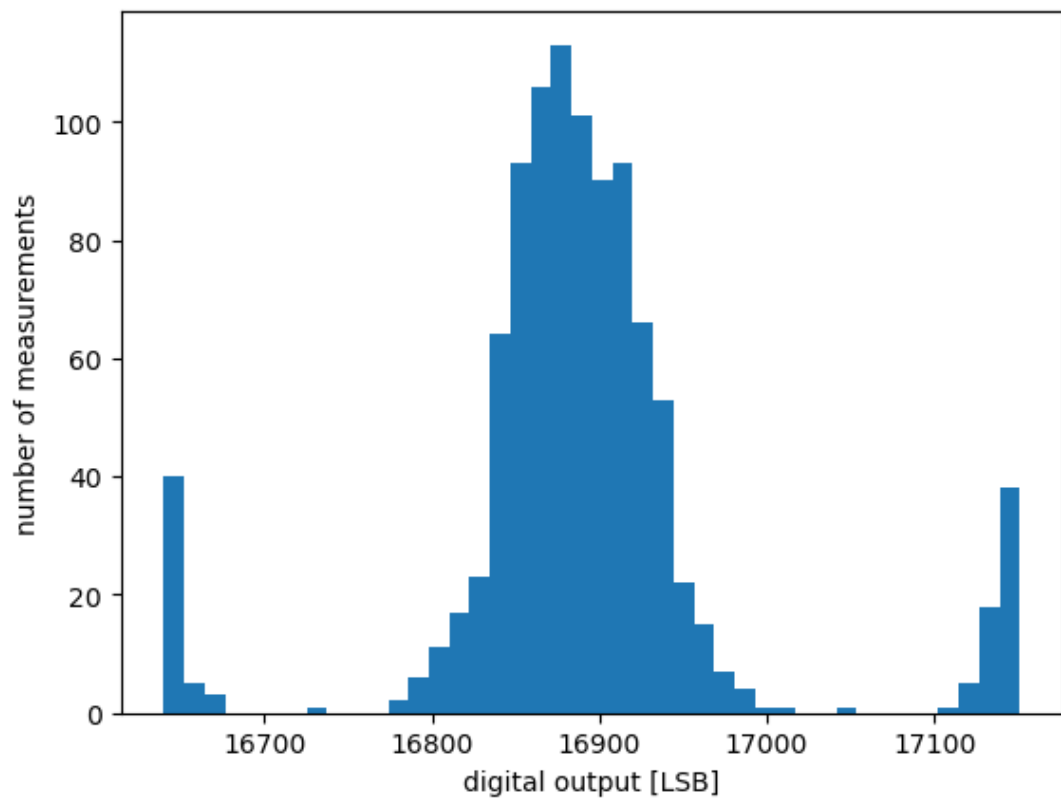


Figure 16: Raw histogram for 6660 Hz

The histograms for 13 Hz and 104 Hz are narrow, which confirms that the signal is stable at low ODR settings. The histograms for higher ODR settings are wider. In particular, the 833 Hz data contains values far away from the main group of measurements. These values are interpreted as outliers or short disturbances.

4.4 Software Filter

A median filter was used to reduce short spikes and outliers in the measured data. A median filter replaces each value with the median value of a small window around this point. This filter is useful for removing short disturbances because a single extreme value does not strongly influence the median.

The implemented filter was:

```
1 from scipy.signal import medfilt
2
3 median_filtered_data = medfilt(raw_data, kernel_size=5)
```

Listing 3: Median filter used for the accelerometer data.

A kernel size of 5 was selected. This means that every filtered output value is calculated from a window of five neighbouring samples. The filter reduces short spikes while keeping the general signal level almost unchanged.

A moving average filter was also tested, but it was not used for the final evaluation. The reason is that the moving average implementation can introduce edge effects at the beginning and end of the signal. This can artificially increase the calculated standard deviation and would therefore not represent the sensor noise correctly.

4.5 Results After Median Filtering

Table 3 shows the results after applying the median filter.

Table 3: Median-filtered accelerometer measurement results.

ODR [Hz]	Samples	Mean [LSB]	Median [LSB]	Min/Max [LSB]	Std. dev. [LSB]
13	1000	16883.56	16884.00	16872 / 16894	3.66
104	1000	16887.24	16887.00	16876 / 16901	3.85
833	1000	16882.71	16881.00	16774 / 17684	47.60
3330	1000	16882.50	16881.00	16827 / 16947	19.51
6660	1000	16885.90	16886.00	16649 / 17137	28.55

The filtered results show that the median filter reduces the influence of short spikes. The effect is small at 13 Hz because the raw signal is already stable. At higher ODR settings, especially at 833 Hz, 3330 Hz and 6660 Hz, the reduction is much stronger.

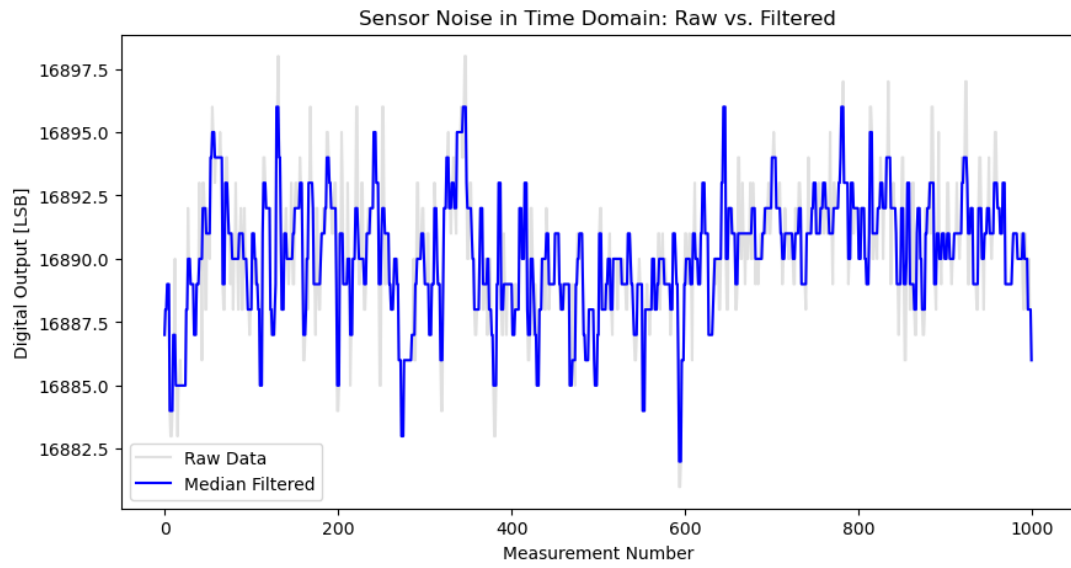


Figure 17: Filtered time-series for 13 Hz

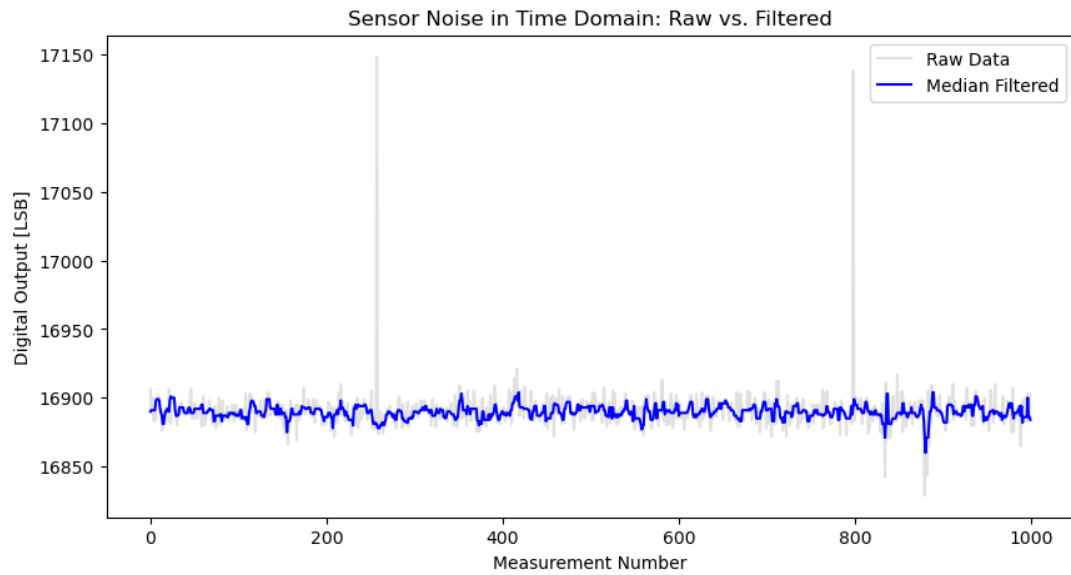


Figure 18: Filtered time-series for 104 Hz

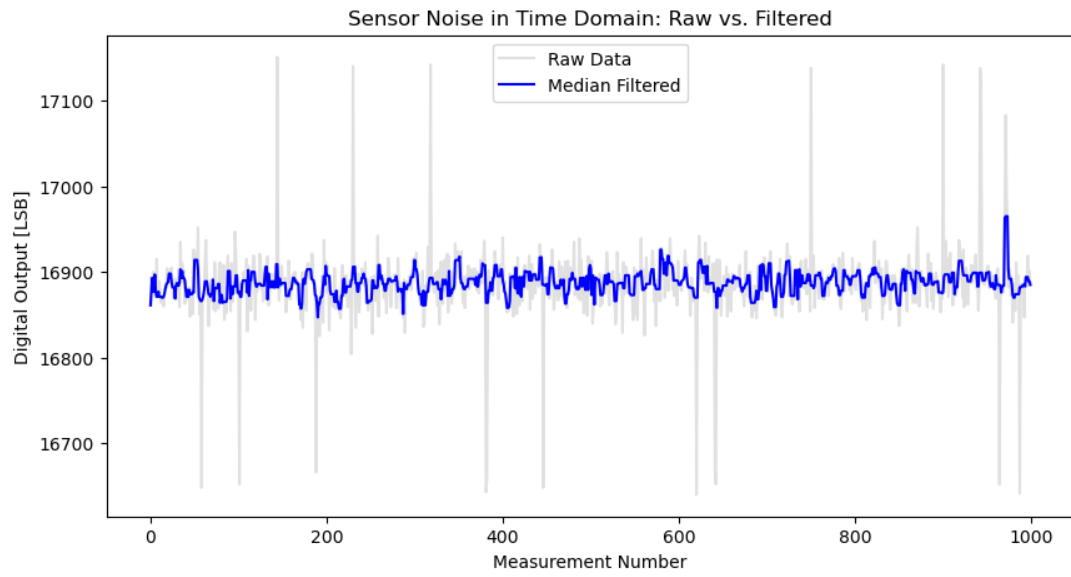


Figure 19: Filtered time-series for 833 Hz

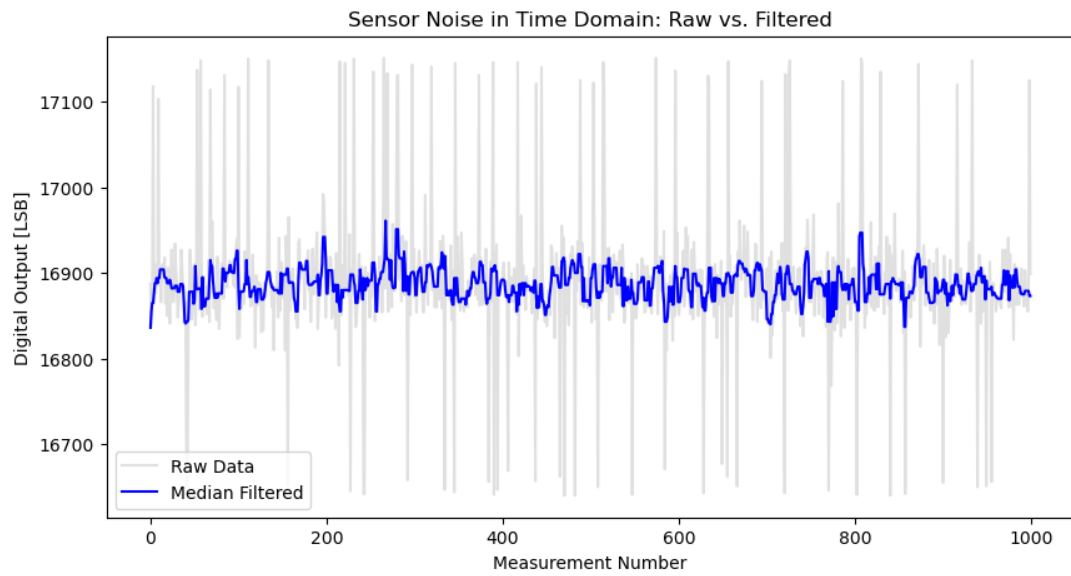


Figure 20: Filtered time-series for 3330 Hz

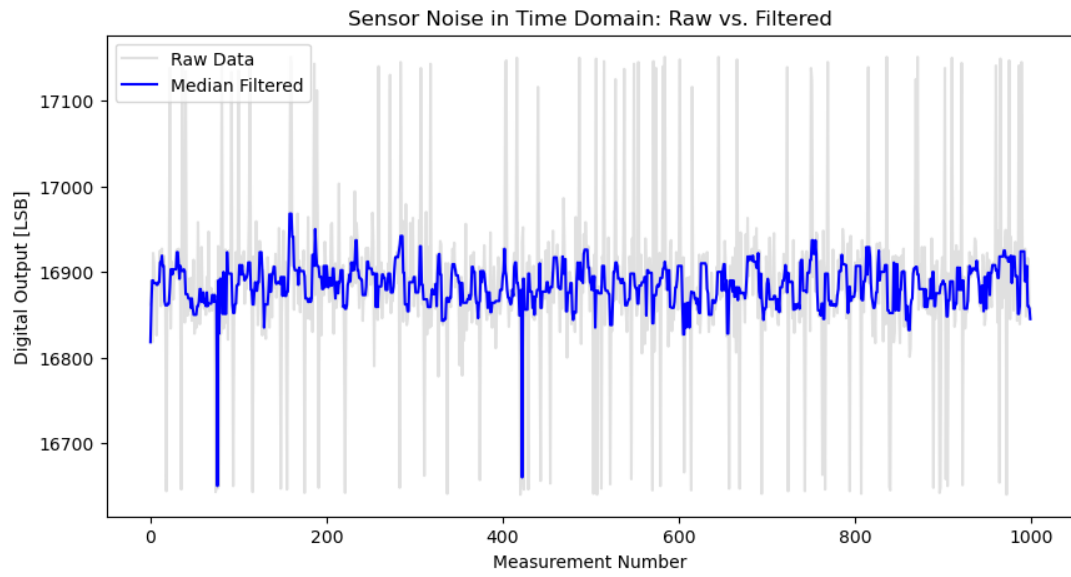


Figure 21: Filtered time-series for 6660 Hz

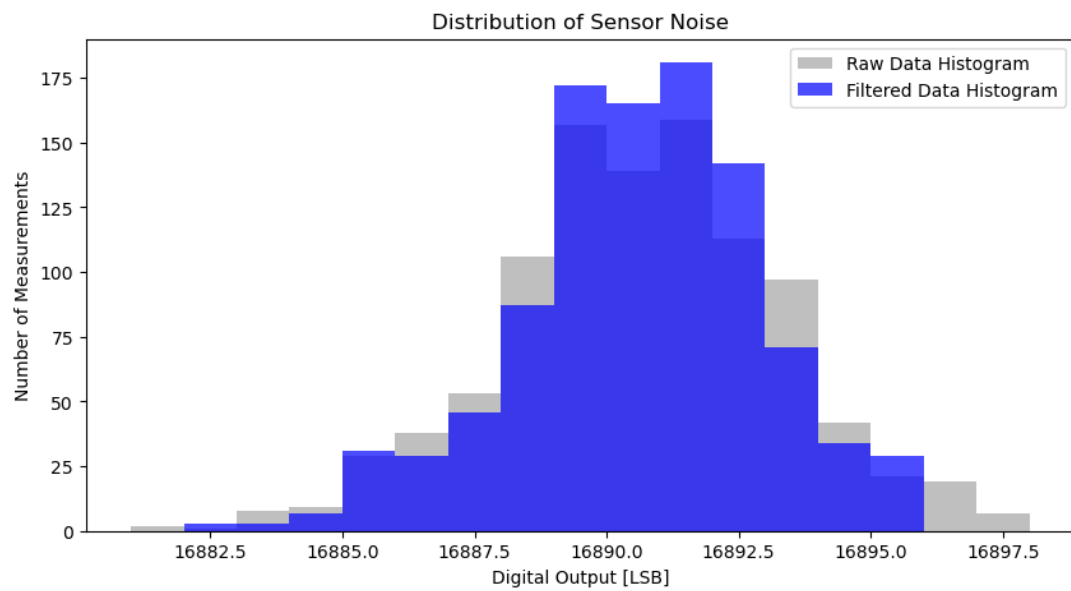


Figure 22: Filtered histogram for 13 Hz

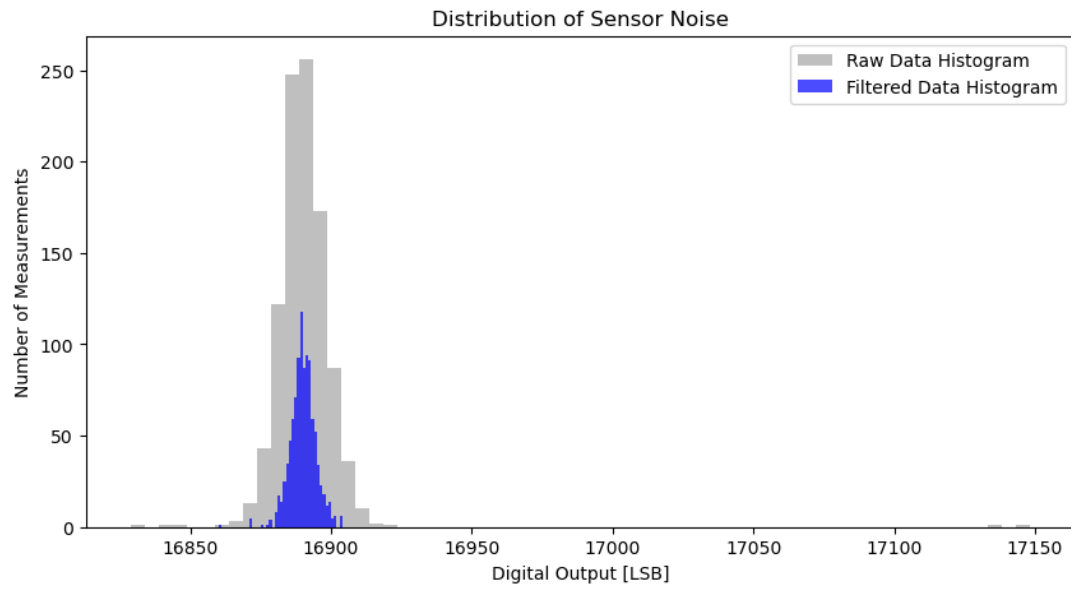


Figure 23: Filtered histogram for 104 Hz

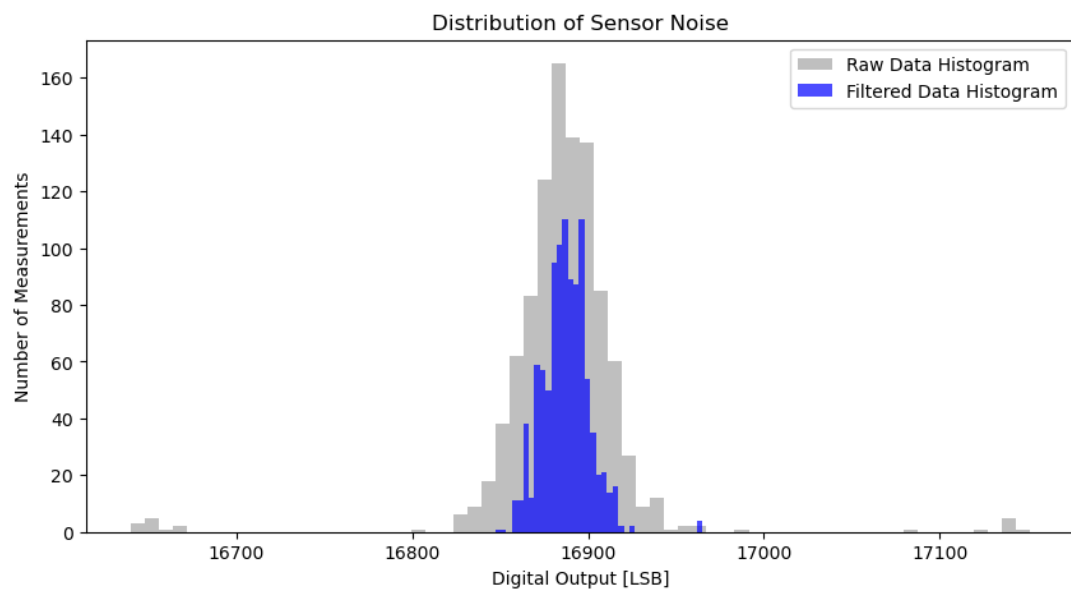


Figure 24: Filtered histogram for 833 Hz

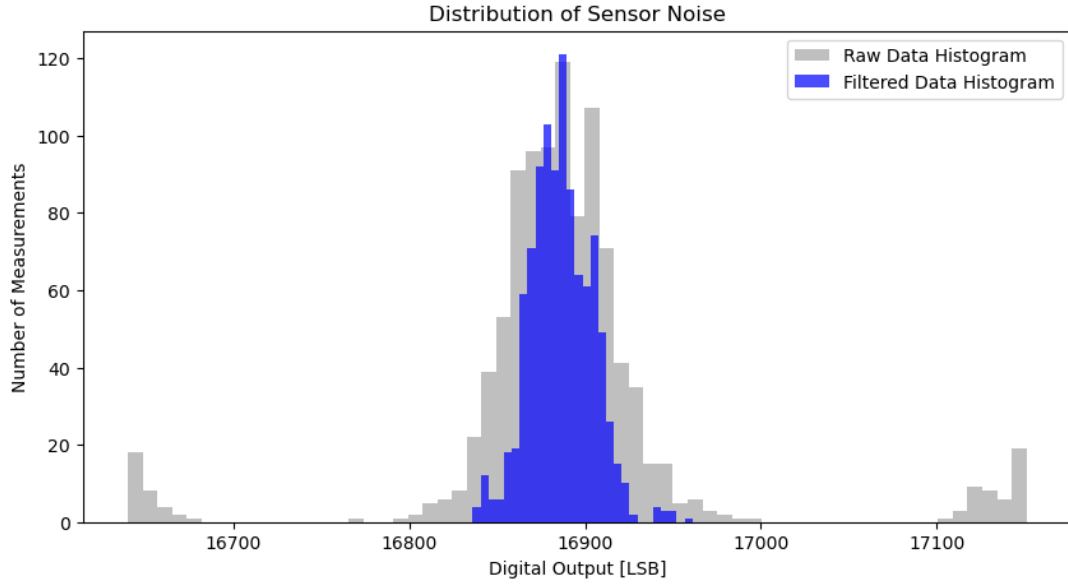


Figure 25: Filtered histogram for 3330 Hz

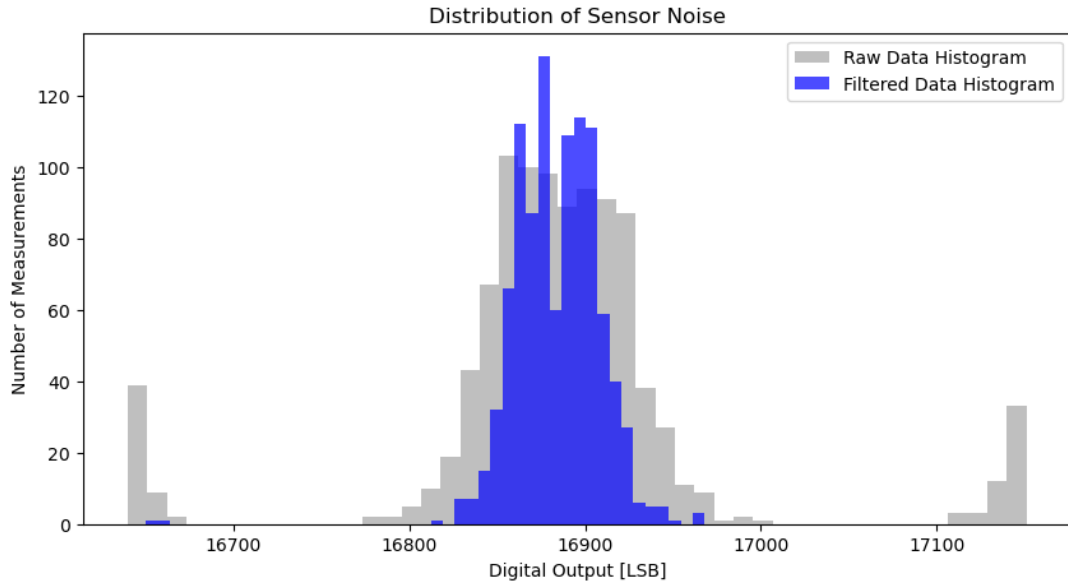


Figure 26: Filtered histogram for 6660 Hz

4.6 Comparison Before and After Filtering

Table 4 compares the raw and filtered standard deviations.

Table 4: Comparison of raw and median-filtered standard deviation.

ODR [Hz]	Raw std. dev. [LSB]	Filtered std. dev. [LSB]	Reduction [%]
13	3.86	3.66	5.14
104	7.25	3.85	46.93
833	161.88	47.60	70.60
3330	70.85	19.51	72.46
6660	89.27	28.55	68.01

The comparison shows that the median filter is effective for removing short disturbances. The standard deviation is reduced for every tested ODR setting. The strongest relative reduction occurs at 3330 Hz, where the standard deviation decreases from 70.85 LSB to 19.51 LSB.

4.7 Standard Deviation as a Function of ODR

Figure 27 shows the standard deviation as a function of the ODR setting. This graph is important because it shows how the noise level changes when the output data rate is increased.

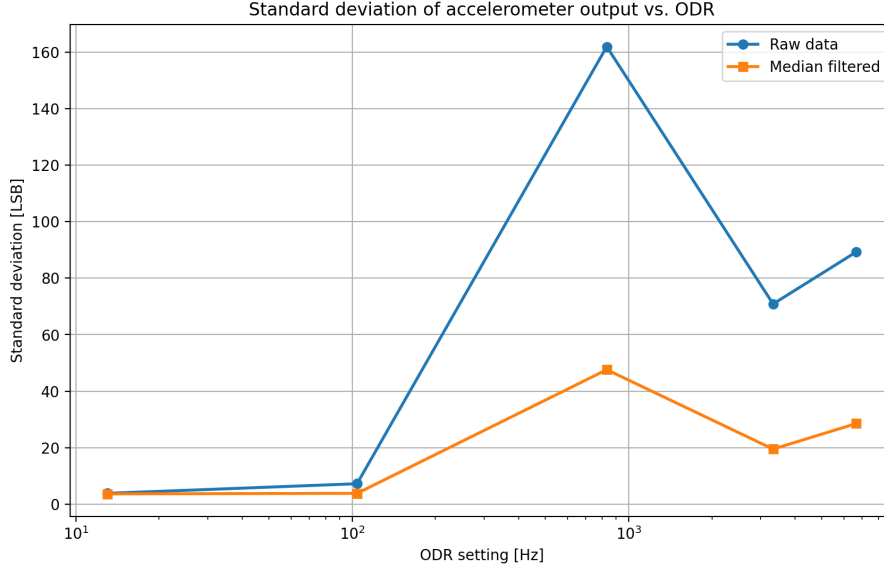


Figure 27: Standard deviation of the accelerometer output as a function of ODR setting.

The raw standard deviation does not increase perfectly monotonically with the ODR setting. The 833 Hz measurement shows an especially high standard deviation because strong outliers are present in the raw data. In general, higher ODR settings can show more noise because the sensor bandwidth is higher and more high-frequency noise components are included in the measured signal. Small mechanical vibrations, electrical noise, sampling effects and short communication disturbances can also become more visible.

After applying the median filter, the standard deviation is reduced for all ODR settings. The filtered values better represent the underlying sensor noise after the removal of short spikes.

4.8 Sensor Offset Calculation

For the $\pm 2g$ measurement range, the ideal digital output for $+1g$ is approximately:

$$x_{\text{ideal},+1g} = 16384 \text{ LSB}$$

The sensor offset was calculated relative to this ideal $+1g$ value:

$$\text{offset}_{LSB} = \bar{x} - 16384$$

$$\text{offset}_g = \frac{\bar{x} - 16384}{16384}$$

where $\bar{x}$ is the mean value of the raw measurement.

Table 5: Calculated accelerometer offset for each ODR setting.

ODR [Hz]	Mean [LSB]	Offset [LSB]	Offset [g]
13	16883.60	499.60	0.03049
104	16887.37	503.37	0.03072
833	16885.58	501.58	0.03061
3330	16881.09	497.09	0.03034
6660	16890.37	506.37	0.03091

The calculated offset is approximately $0.030g$ for all ODR settings. This means that the measured value is around 3% higher than the ideal $+1g$ value. The offset is very similar for all tested ODR settings. Therefore, the offset is probably not caused by the selected ODR. Possible reasons are a small sensor bias, imperfect alignment of the sensor with the gravity vector, mounting tolerances, or calibration error.

No unexpected systematic change of the offset with ODR was observed. The largest differences between the settings are caused by noise and outliers, not by a clear drift of the mean value.

4.9 Discussion

The results show that the accelerometer output is very stable at low ODR settings. At 13 Hz, the standard deviation is only 3.86 LSB. At 104 Hz, it increases to 7.25 LSB. At higher ODR settings, the measured signal becomes more sensitive to noise and short disturbances.

The 833 Hz measurement contains the strongest outliers. This explains the high raw standard deviation of 161.88 LSB. After applying the median filter, the standard deviation is reduced to 47.60 LSB. This confirms that a large part of the high raw standard deviation was caused by short spikes rather than by a permanent change of the signal.

The median filter is suitable for this measurement because it removes short disturbances while keeping the general signal level almost unchanged. The filtered data still shows higher noise at high ODR settings than at low ODR settings, but the influence of single extreme values is clearly reduced.

Overall, the measurement confirms that the selected ODR setting has a clear influence on the observed noise behaviour. Lower ODR settings provide a more stable output, while higher ODR settings include more high-frequency noise and are more sensitive to vibrations and short disturbances.

5 Part 5 – Noise on a yaw rate sensor

5.1 Aim of the measurement

The aim of this part was to analyse the noise behaviour of one gyroscope axis of the LSM6DS3 inertial measurement unit. The measurements were performed for two gyroscope measurement ranges, 245 dps and 500 dps, and for four ODR/bandwidth settings: 13 Hz, 104 Hz, 416 Hz, and 1.66 kHz. During all measurements the sensor was kept as still as possible, so the expected angular rate was approximately 0 dps. Therefore, any non-zero average output can be interpreted as the sensor offset.

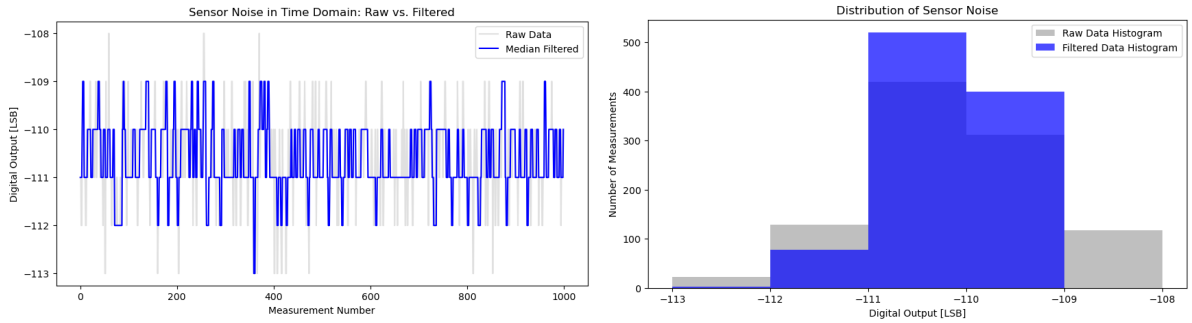
For every setting, 1000 samples were recorded. The raw time-domain signal was first checked for fluctuations and possible spikes. Afterwards, a median filter was applied in software and the filtered result was compared with the raw result. The measured values are given in digital output counts, also called LSB. For interpretation in physical units, the gyroscope sensitivities used were:

- 245 dps range: $8.75 \text{ mdps/LSB} = 0.00875 \text{ dps/LSB}$
- 500 dps range: $17.50 \text{ mdps/LSB} = 0.01750 \text{ dps/LSB}$

5.2 Time series and histograms

The following figures show the raw and median-filtered time series as well as the corresponding histograms. The raw signal represents the original gyroscope output. The filtered signal represents the output after applying the median filter.

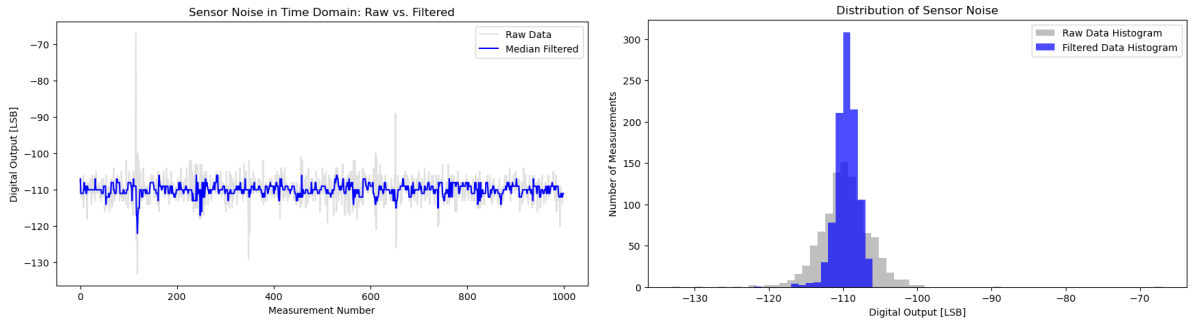
5.2.1 Measurement range: 245 dps



Time series, 13 Hz, 245 dps

Histogram, 13 Hz, 245 dps

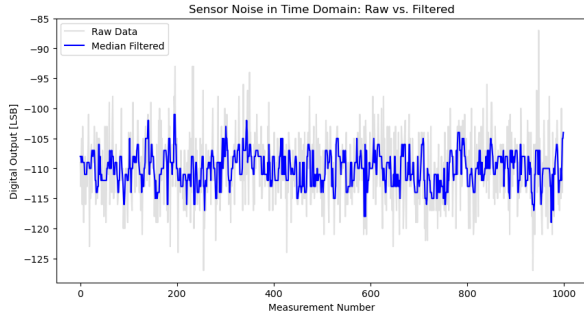
Figure 28: Gyroscope noise at 13 Hz ODR and 245 dps range.



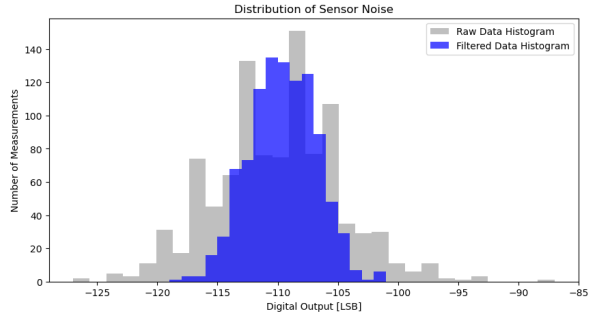
Time series, 104 Hz, 245 dps

Histogram, 104 Hz, 245 dps

Figure 29: Gyroscope noise at 104 Hz ODR and 245 dps range.

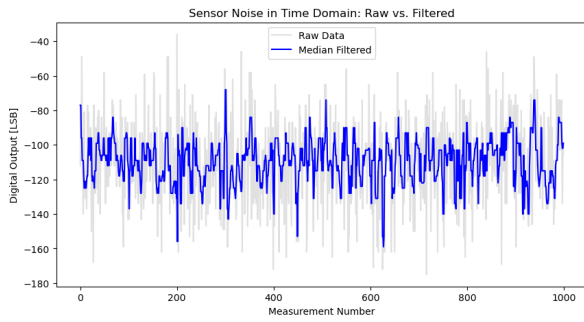


Time series, 416 Hz, 245 dps

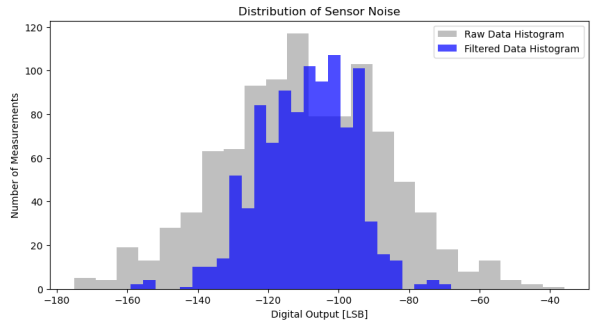


Histogram, 416 Hz, 245 dps

Figure 30: Gyroscope noise at 416 Hz ODR and 245 dps range.



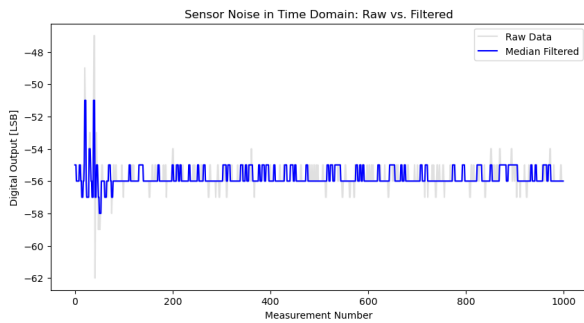
Time series, 1.66 kHz, 245 dps



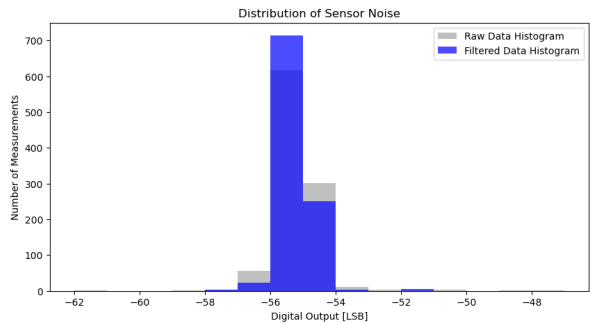
Histogram, 1.66 kHz, 245 dps

Figure 31: Gyroscope noise at 1.66 kHz ODR and 245 dps range.

5.2.2 Measurement range: 500 dps

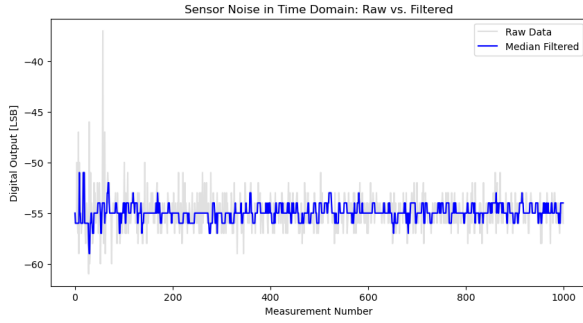


Time series, 13 Hz, 500 dps

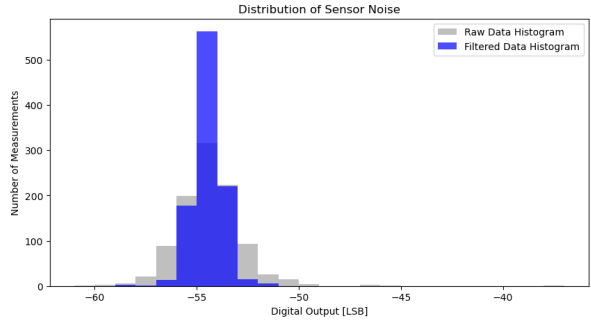


Histogram, 13 Hz, 500 dps

Figure 32: Gyroscope noise at 13 Hz ODR and 500 dps range.

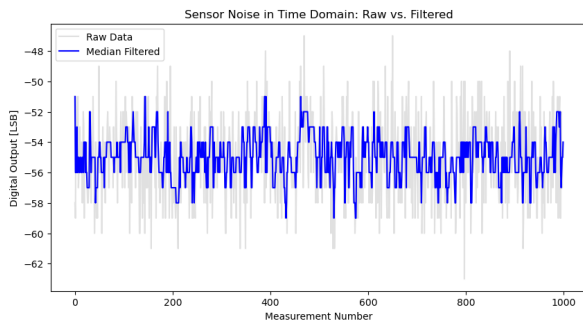


Time series, 104 Hz, 500 dps

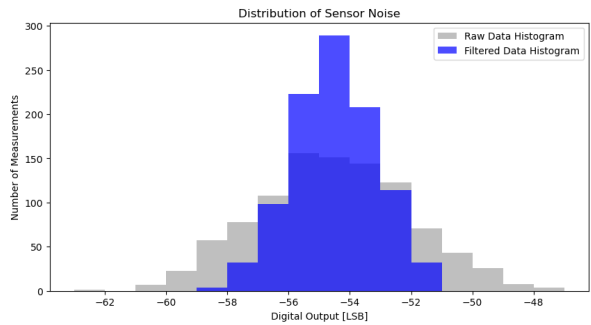


Histogram, 104 Hz, 500 dps

Figure 33: Gyroscope noise at 104 Hz ODR and 500 dps range.

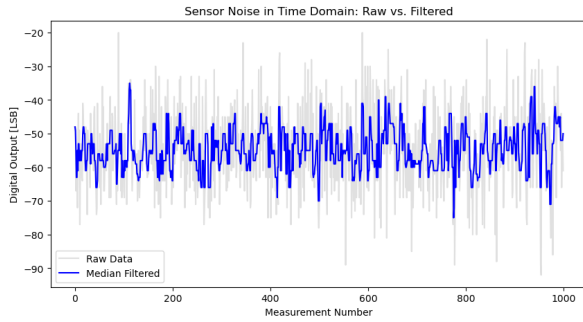


Time series, 416 Hz, 500 dps

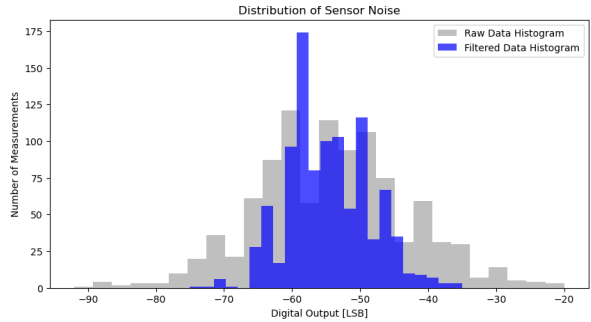


Histogram, 416 Hz, 500 dps

Figure 34: Gyroscope noise at 416 Hz ODR and 500 dps range.



Time series, 1.66 kHz, 500 dps



Histogram, 1.66 kHz, 500 dps

Figure 35: Gyroscope noise at 1.66 kHz ODR and 500 dps range.

5.3 Statistical results

Table 6: Statistical results for the gyroscope noise measurements.

Range [dps]	ODR [Hz]	Raw mean [LSB]	Raw median [LSB]	Raw std [LSB]	Median-filter std [LSB]	Offset [dps]
245	13	-110.62	n/a	0.94	0.73	-0.968
245	104	-109.89	n/a	3.63	1.56	-0.962
245	416	-110.02	n/a	5.18	2.77	-0.963
245	1660	-109.86	n/a	22.87	12.88	-0.961
500	13	-55.70	n/a	0.83	0.62	-0.975
500	104	-54.87	n/a	1.62	0.80	-0.960
500	416	-55.02	n/a	2.49	1.40	-0.963
500	1660	-54.50	n/a	11.13	6.06	-0.954

The exported measurement files contained the printed mean and standard deviation values. The exact median values were not printed in the exported output. Therefore, they are marked as n/a in Table 6. The Python code in Listing 4 shows how the median values can be calculated directly from the raw measurement vector using `np.median(a)`.

5.4 Standard deviation as a function of ODR

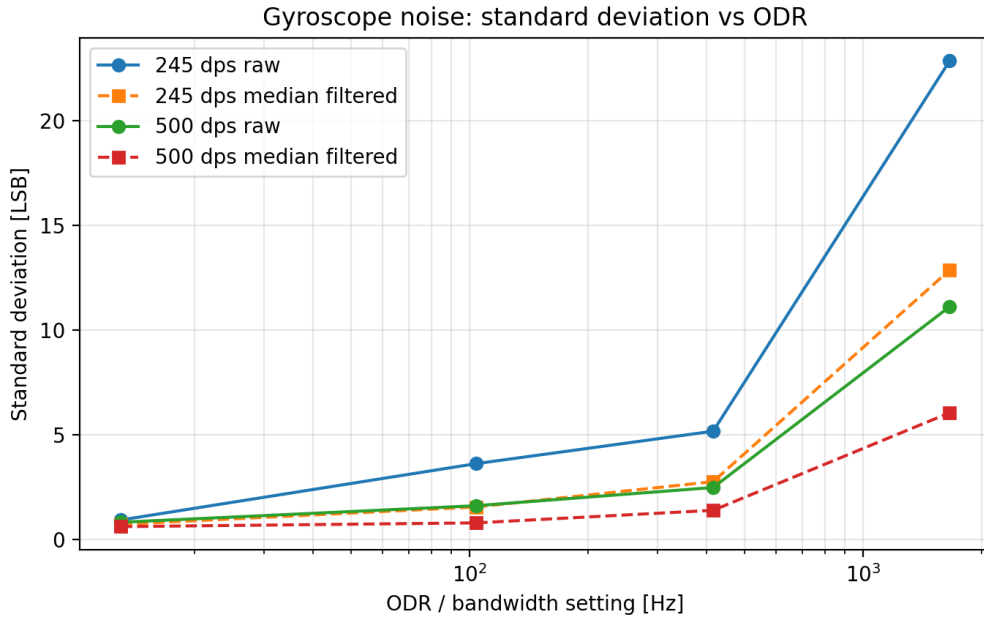


Figure 36: Standard deviation of the gyroscope output as a function of ODR/bandwidth setting for 245 dps and 500 dps, before and after median filtering.

The standard deviation increases with increasing ODR. This behaviour is expected because a higher ODR/bandwidth setting allows more high-frequency noise components to pass through the sensor signal path. At the lowest ODR, the output is concentrated in a narrow range of digital values. At 1.66 kHz, the time-series plots show much larger fluctuations and the histograms become wider. This means that the signal is noisier even though the sensor is not intentionally moving.

The median filter reduces the standard deviation for all tested settings. The reduction is especially visible at the higher ODR settings. This indicates that part of the measured noise consists of short spikes or fast variations, which are suppressed by the median filter.

5.5 Sensor offset calculation

Since the gyroscope was stationary during the measurement, the expected angular rate was:

$$\omega_{\text{expected}} = 0 \text{ dps}$$

Therefore, the sensor offset was calculated from the mean value of the raw digital output:

$$\text{offset}_{\text{LSB}} = \bar{x}_{\text{raw}}$$

The physical offset in degrees per second was then calculated using the sensitivity of the selected measurement range:

$$\text{offset}_{\text{dps}} = \bar{x}_{\text{raw}} \cdot S$$

where S is the sensitivity in dps/LSB. For the 245 dps range, $S = 0.00875$ dps/LSB. For the 500 dps range, $S = 0.01750$ dps/LSB.

For the 245 dps range, the raw mean value was approximately -110 LSB. This corresponds to an offset of approximately -0.96 to -0.97 dps. For the 500 dps range, the raw mean value was approximately -55 LSB. Because the sensitivity is twice as large in this range, this also corresponds to approximately -0.95 to -0.98 dps.

This is a useful consistency check. The digital output changes with the selected measurement range, but the calculated physical offset remains almost the same. This means that the measurements are consistent. In practical applications, this static offset of approximately -0.96 dps must be subtracted from the raw readings in software. If left uncorrected, mathematical integration of the signal to calculate angular position would result in an accumulated drift of nearly 1 degree every second, rapidly destabilizing any orientation-based control system.

5.6 Filter documentation

The main software filter used in this part was a median filter with a kernel size of 5 samples. A median filter replaces each sample by the median value of a small window around that sample. This is useful for IMU noise measurements because isolated spikes or very short disturbances are removed without strongly shifting the centre value of the signal.

```
1 import numpy as np
2 from scipy.signal import medfilt
3
4 # a contains the raw gyroscope data in LSB
5 # Example:
6 # a = np.array(daten)
7
8 # Median filter for spike/outlier reduction
9 median_filtered_data = medfilt(a, kernel_size=5)
10
11 # Optional moving average filter for comparison
12 window_size = 5
13 ma_filtered_data = np.convolve(
14     a,
15     np.ones(window_size) / window_size,
16     mode='same'
17 )
18
19 # Statistical values for the report
20 raw_mean = np.mean(a)
21 raw_median = np.median(a)
22 raw_std = np.std(a)
23
24 median_mean = np.mean(median_filtered_data)
```

```

25 median_median = np.median(median_filtered_data)
26 median_std = np.std(median_filtered_data)
27
28 ma_std = np.std(ma_filtered_data)
29
30 print(f"RAW Mean: {raw_mean:.2f}")
31 print(f"RAW Median: {raw_median:.2f}")
32 print(f"RAW Std Dev: {raw_std:.2f}")
33
34 print(f"Median Filter Mean: {median_mean:.2f}")
35 print(f"Median Filter Median: {median_median:.2f}")
36 print(f"Median Filter Std Dev: {median_std:.2f}")
37
38 print(f"Moving Average Std Dev: {ma_std:.2f}")

```

Listing 4: Python filtering and statistics code used for the gyroscope noise analysis.

5.7 Conclusion

The gyroscope noise clearly depends on the selected ODR/bandwidth. The lowest tested setting, 13 Hz, produced the smallest standard deviation, while the highest setting, 1.66 kHz, produced the largest fluctuations. This is visible both in the time-domain plots and in the wider histograms at higher ODR values.

The median filter reduced the standard deviation in all cases, confirming that software filtering can improve the stability of the output when the sensor is stationary. The calculated sensor offset was approximately -0.96 dps for both measurement ranges. In raw LSB values, the offset was about -110 LSB for the 245 dps range and about -55 LSB for the 500 dps range. This difference is caused by the different sensitivity of the two ranges. After conversion to dps, the offset is nearly the same, which means the measurements are consistent.

6 Filter Implementation Source Code

The following Python script was utilized to acquire data from the serial port, log it to a CSV file, and apply the median filter algorithm to mitigate sensor noise.

```

1 # -- coding: utf-8 --
2 import serial
3 import csv
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from scipy.signal import medfilt
7
8 ser = serial.Serial('/dev/cu.usbserial-1470', 19200)
9 ser.flushInput()
10 max_measurements = 1000
11 daten = []
12 x = np.arange(0, max_measurements, 1)
13
14 for n in range(0, 20):
15     ser.readline()
16
17 print(f"Collecting {max_measurements} measurements... Please keep the sensor
18     still.")
19
20 for n in range(0, max_measurements):
21     ser_bytes = ser.readline()
22     try:
23         decoded_bytes = int(ser_bytes.decode("utf-8").strip())
24         daten.append(decoded_bytes)

```

```

24
25         with open("filename.csv", "a", newline='') as f:
26             writer = csv.writer(f, delimiter=" ")
27             writer.writerow([decoded_bytes])
28     except ValueError:
29         daten.append(0)
30
31 ser.close()
32 print("Data collection complete. Processing filters...")
33
34 a = np.hstack(daten)
35
36 # Median Filter
37 median_filtered_data = medfilt(a, kernel_size=5)
38
39 print("\n--- STATISTICAL ANALYSIS ---")
40 print(f"RAW Mean: {np.mean(a):.2f} | RAW Std Dev: {np.std(a):.2f}")
41 print(f"MEDIAN Filter Std Dev: {np.std(median_filtered_data):.2f}")

```

Listing 5: Python Script for Data Logging and Filtering